

PROJECT MS

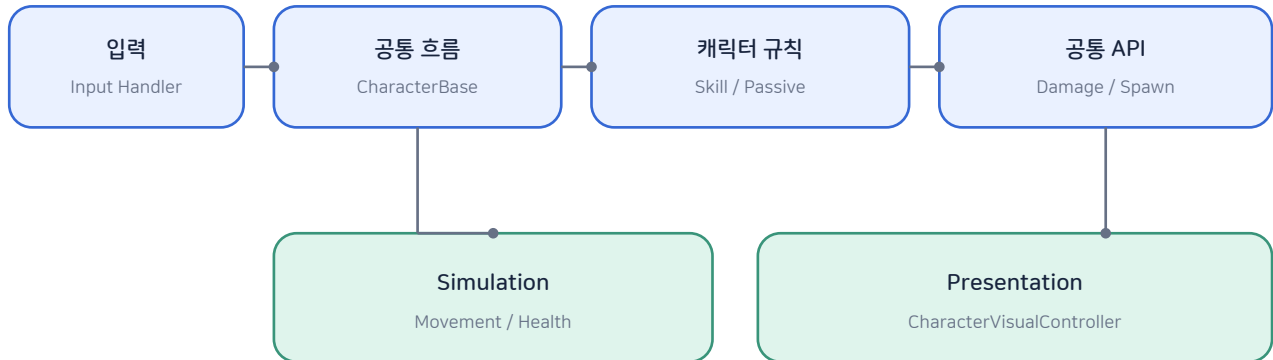
신규 캐릭터 개발 가이드

공통 구조를 이해하고 스킬과 패시브 구현에 집중하는
Photon Fusion 2 Shared Mode 제작 가이드

대단원별 페이지 분리 | 세부 내용은 한 흐름으로 연결

한눈에 보는 구조

공통 시스템은 입력부터 네트워크 표현까지의 흐름을 책임지고, 캐릭터별 클래스는 전투 규칙만 구현합니다.



네트워크와 공통 처리는 Base가 맡고, 캐릭터 코드는 규칙에만 집중한다.

핵심 계약

캐릭터별 스크립트에는 Update, FixedUpdateNetwork, Networked, Rpc, Runner.Spawn을 직접 작성하지 않습니다.

문서 목적

이 문서는 현재 캐릭터 공통 프레임워크의 구조와 신규 캐릭터를 만드는 절차만 설명한다.

대상: 신규 캐릭터의 스킬과 패시브를 구현하는 개발자

네트워크 기준: Photon Fusion 2, GameMode.Shared

목표: 캐릭터 제작자는 Fusion 내부 구현보다 캐릭터의 전투 규칙에 집중한다.

범위: 공통 구조, 프리팹 구성, 데이터 설정, Character API, 스킬과 패시브 구현, 검수 절차

1. 한눈에 보는 전체 구조

캐릭터 입력부터 화면 표현까지의 흐름은 다음과 같다.

```

로컬 입력
↓
CharacterInputHandler
↓
CharacterBase.FixedUpdateNetwork
↓
캐릭터별 스킬과 패시브
↓
CharacterBase 공통 API
↓
동기화된 상태와 비주얼 이벤트
↓
CharacterVisualController
  
```

CharacterBase가 입력 권한, 상태 권한, 쿨타임, 체력, 네트워크 동기화를 처리한다. 신규 캐릭터 클래스는 정해진 스킬 함수와 패시브 이벤트만 구현한다.

역할 분리

영역	책임	신규 캐릭터 개발자가 수정하는가?
CharacterBase	네트워크 실행 흐름, 공통 전투 API, 상태 동기화	아니요
CharacterInputHandler	입력 수집과 짧은 입력 보관	아니요
CharacterMovementHandler	이동, 점프, 대시, 지면 판정	보통 수정하지 않음
CharacterHealth	체력과 사망 처리	아니요
CharacterCooldownHandler	행동별 쿨타임 처리	아니요
CharacterVisualController	공통 움직임과 피격 표현	설정만 연결
캐릭터별 클래스	평타, Q, E, 궁극기, 패시브	예
CharacterDefinition	체력, 이동, 데미지, 쿨타임 수치	예
CharacterVisualProfile	공통 비주얼 반응 수치	필요한 경우만

2. 폴더와 주요 파일

```

Character
├── Runtime
│   ├── Core
│   │   ├── CharacterBase.cs
│   │   ├── CharacterActionContext.cs
│   │   └── CharacterSockets.cs
│   ├── Data
│   │   ├── CharacterDefinition.cs
│   │   └── CharacterVisualProfile.cs
│   ├── Modules
│   │   ├── CharacterInputHandler.cs
│   │   ├── CharacterMovementHandler.cs
│   │   ├── CharacterHealth.cs
│   │   └── CharacterCooldownHandler.cs
│   ├── Combat
│   │   ├── CharacterCombatQuery2D.cs
│   │   └── CharacterProjectile.cs
│   ├── Visual
│   │   ├── CharacterVisualController.cs
│   │   └── Solver
│   └── Examples
│       └── CharacterTemplate.cs
├── Editor
│   └── CharacterPrefabValidator.cs
└── Docs
  
```

신규 캐릭터를 만들 때 주로 다루는 것은 캐릭터별 스크립트, **CharacterDefinition**, 프리팹 Variant 세 가지다. 공통 Core, Modules, Combat, Visual 코드는 공통 시스템 담당자와 협의하지 않고 직접 변경하지 않는다.

3. 캐릭터 프리팹 구조

모든 캐릭터는 동일한 기본 프리팹 구조를 사용한다.

```

CharacterRoot
├─ 캐릭터별 스크립트 : CharacterBase 상속
├─ Rigidbody2D
├─ Collider2D
├─ NetworkObject
├─ 프로젝트 표준 NetworkRigidbody2D
├─
├─ VisualRoot
├─ CharacterVisualController
├─ Body
├─ AttackOrigin
├─ ProjectileOrigin   선택
├─ WeaponRoot        선택
├─ EffectOrigin       선택
├─ WorldCanvas        선택
├─ DustEffect         선택
  
```

Socket 사용 기준

Socket	용도	예시
Attack Origin	근접 또는 범위 판정의 기준 위치	검, 주먹, 부채꼴 공격
Projectile Origin	투사체 생성 위치	총알, 화살, 마법탄
Weapon Root	무기 비주얼의 부모 Transform	검, 총, 활, 지팡이
Effect Origin	VFX 생성의 기본 위치	폭발, 섬광, 타격 효과

사용하지 않는 Socket은 비워둘 수 있다. 검 캐릭터에 **ProjectileOrigin**을 억지로 만들거나, 무기가 없는 캐릭터에 **WeaponRoot**를 추가할 필요는 없다.

컴포넌트 수를 늘리지 않는 원칙

공통 움직임, 조준 표현, Squash와 Stretch, 피격 플래시, Jiggle은 **CharacterVisualController** 한 곳에서 설정한다. 캐릭터마다 작은 Visual MonoBehaviour를 추가하지 않고, Profile과 대상 Transform 목록으로 차이를 표현한다.

4. 캐릭터 데이터 만들기

Character Definition

Unity Project 창에서 다음 메뉴로 생성한다.

Create > Project MS > Character > Definition

CharacterDefinition에는 캐릭터의 기본 수치를 설정한다.

표시 이름과 최대 체력

이동 속도, 가속도, 점프 힘

지면 판정 Layer

점프 버퍼와 코요테 타임

대시 힘과 지속 시간

Auto Hop 사용 여부와 수치

평타, Q, E, 궁극기 데미지와 쿨타임

기본 입력 키

실행 중 변하는 현재 체력, 남은 쿨타임, 버프 상태는 Definition에 저장하지 않는다.

Character Visual Profile

기본적으로 팀 공통 Profile을 사용한다. 캐릭터의 무게감이나 탄성이 확실히 달라야 할 때만 별도 Profile을 생성한다.

Create > Project MS > Character > Visual Profile

Profile에서 조정할 수 있는 항목은 점프 Stretch, 착지 Squash, 대기 Wobble, 피격 플래시, Jiggle 복원력과 감쇠 등이다.

5. 캐릭터별 스크립트 구현

CharacterTemplate.cs를 복사하고 클래스와 파일 이름을 캐릭터 이름으로 변경한다. 필요한 함수만 override한다.

```
public sealed class NewCharacter : CharacterBase
{
    protected override bool OnBasicAttack(CharacterActionContext context)
    {
        // 평타 규칙
        return true;
    }

    protected override bool OnSkillQ(CharacterActionContext context)
    {
        // Q 스킬 규칙
        return true;
    }

    protected override bool OnSkillE(CharacterActionContext context)
    {
        // E 스킬 규칙
        return true;
    }

    protected override bool OnUltimate(CharacterActionContext context)
    {
        // 궁극기 규칙
        return true;
    }
}
```

스킬이 정상적으로 실행되었으면 **true**, 실행 조건이 맞지 않아 취소되었으면 **false**를 반환한다. 공통 흐름은 성공한 행동에 대해서만 쿨타임과 후속 이벤트를 처리한다.

공격 방식에 맞는 공통 API 선택

무기 종류에 따라 상속 구조를 다시 만들지 않는다. 스킬 안에서 판정 API만 선택한다.

구현하려는 동작	사용하는 API
총알, 화살, 마법탄	SpawnProjectile
검이나 부채꼴 근접 공격	FindEnemiesInArc + DealDamage
자신 중심 범위 공격	FindEnemiesInCircle + DealDamage
직선 관통 공격	FindEnemiesInLine + DealDamage
사각형 범위 공격	FindEnemiesInBox + DealDamage
기본 대시	StartDefaultDash
방향과 힘이 다른 대시	StartDash
공통 공격 VFX	PlayActionEffect

CharacterActionContext에서 조준 방향, 조준 각도, 공격 기준 위치, 데미지 등 현재 행동에 필요한 값을 가져온다.

패시브 구현

패시브는 매 프레임 직접 검사하지 않고 상황에 맞는 이벤트를 우선 사용한다.

발생 조건	override 함수
피격됨	OnDamaged
다른 캐릭터에게 피해를 줌	OnDamageDealt
사망함	OnDied
점프함	OnJumped
착지함	OnLanded
스킬 실행에 성공함	OnSkillExecuted
체력이 변경됨	OnHealthChanged
지속적으로 확인해야 함	OnPassiveTick

예를 들어 피격 시 방어력이 오르는 패시브는 OnDamaged, 착지 시 충격파가 발생하는 패시브는 OnLanded에 구현한다. 이벤트로 표현할 수 없는 지속 조건에만 OnPassiveTick을 사용한다.

6. 캐릭터 제작용 API 안내

이 절에 있는 API는 `CharacterBase`를 상속한 캐릭터 클래스에서 사용한다. 네트워크 권한과 실행 위치는 Base가 처리하므로 캐릭터 코드는 아래 API의 게임 규칙만 작성한다.

6.1 스킬 진입점

입력과 쿨타임 검사를 통과하면 Base가 해당 함수를 호출한다.

API	호출 시점	반환값
<code>OnBasicAttack(context)</code>	기본 공격 입력	실행 성공 여부
<code>OnSkillQ(context)</code>	Q 입력	실행 성공 여부
<code>OnSkillE(context)</code>	E 입력	실행 성공 여부
<code>OnDash(context)</code>	대시 입력	실행 성공 여부
<code>OnUltimate(context)</code>	궁극기 입력	실행 성공 여부

행동이 실제로 실행되었으면 `true`를 반환한다. 거리나 자원 같은 추가 조건이 맞지 않아 행동을 취소했으면 `false`를 반환한다. `false`를 반환한 행동에는 공통 쿨타임과 성공 이벤트가 적용되지 않는다.

6.2 CharacterActionContext

모든 스킬 진입점은 현재 행동에 필요한 값을 `context`로 받는다.

값	타입	의미
<code>context.Action</code>	<code>CharacterActionType</code>	현재 실행 중인 행동 종류
<code>context.Origin</code>	<code>Vector2</code>	공통 시스템이 결정한 공격 기준 위치
<code>context.AimDirection</code>	<code>Vector2</code>	정규화된 조준 방향
<code>context.AimWorldPosition</code>	<code>Vector2</code>	조준 중인 월드 좌표
<code>context.AimAngle</code>	<code>float</code>	조준 방향을 각도로 변환한 값
<code>context.Damage</code>	<code>float</code>	Definition에 설정한 현재 행동의 기본 데미지

캐릭터 스크립트에서 마우스 위치나 입력을 다시 읽지 않고 `context` 값을 사용한다. 그래야 로컬과 원격에서 같은 방향과 결과가 사용된다.

6.3 타겟 검색 API

검색 API는 자기 자신을 제외하고 `targetLayer`에 포함된 캐릭터 목록을 반환한다. 검색 결과에 데미지를 주려면 각 대상에 `DealDamage`를 호출한다.

API	영역	주요 인자
FindEnemiesInCircle	원형 범위	center, radius, targetLayer
FindEnemiesInBox	회전 가능한 사각 범위	center, size, angle, targetLayer
FindEnemiesInLine	폭이 있는 직선 범위	origin, direction, distance, width, targetLayer
FindEnemiesInArc	방향을 가진 부채꼴 범위	origin, direction, radius, angle, targetLayer

FindEnemiesInCircle

캐릭터 중심 폭발, 오라, 착지 충격파처럼 방향이 필요 없는 범위 판정에 사용한다.

```
List targets = FindEnemiesInCircle(
    transform.position,
    radius,
    targetLayer);
```

FindEnemiesInBox

벽, 넓은 검기, 직사각형 장판처럼 가로와 세로 크기가 다른 판정에 사용한다. **angle**은 사각형의 회전 각도다.

```
List targets = FindEnemiesInBox(
    context.Origin,
    boxSize,
    context.AimAngle,
    targetLayer);
```

FindEnemiesInLine

레이저나 직선 관통 공격처럼 시작점부터 일정 거리까지 이어지는 판정에 사용한다. **width**로 선의 두께를 지정한다.

```
List targets = FindEnemiesInLine(
    context.Origin,
    context.AimDirection,
    distance,
    width,
    targetLayer);
```

FindEnemiesInArc

검, 주먹, 부채꼴 브레스처럼 바라보는 방향을 중심으로 펼쳐지는 공격에 사용한다. **radius**는 사거리, **angle**은 전체 부채꼴 각도다.

```
List targets = FindEnemiesInArc(
    context.Origin,
    context.AimDirection,
    radius,
    angle,
    targetLayer);
```

6.4 전투 API

DealDamage

DealDamage(target, amount)는 대상에게 피해를 요청하고 공격자의 **OnDamageDealt** 이벤트를 호출한다. 대상이 없거나 자기 자신이거나 데미지가 0 이하이면 적용하지 않는다.

```
foreach (CharacterBase target in targets)
{
    DealDamage(target, context.Damage);
}
```

Heal

Heal(amount)는 자신의 체력을 회복한다. 실제 체력 변경과 권한 처리는 Base가 담당한다.

```
Heal(healAmount);
```

SpawnProjectile

투사체 **NetworkObject**를 한 번만 생성하고 방향, 속도, 데미지, 대상 Layer를 초기화한다.

```
SpawnProjectile(
    projectilePrefab,
    ProjectileOrigin.position,
    context.AimDirection,
    projectileSpeed,
    context.Damage,
    targetLayer);
```

인자는 순서대로 투사체 프리팹, 생성 위치, 방향, 속도, 데미지, 공격 대상 Layer다. 캐릭터 클래스에서 **Runner.Spawn**을 직접 호출하지 않는다.

6.5 이동과 표현 API

API	용도	주의점
StartDefaultDash()	Definition의 기본 대시 실행	일반적인 대시는 이것을 사용
StartDash(direction, power, duration)	방향, 힘, 시간이 다른 대시	특수 이동 스킬에 사용
PlayActionEffect(action, position, angle)	모든 클라이언트에 행동 VFX 요청	게임 판정에는 사용하지 않음

PlayActionEffect는 결과를 보여주는 Presentation API다. 데미지나 이동 같은 게임 상태는 먼저 처리하고, 그 결과를 표현할 때 호출한다.

6.6 상태와 Socket 조회

속성	의미
Definition	연결된 CharacterDefinition
Visual	연결된 CharacterVisualController
CurrentHealth	현재 체력
MaxHealth	최대 체력
CurrentHealthPercent	0부터 1 사이의 체력 비율
IsDead	사망 상태
Cooldowns	행동별 남은 쿨타임 조회 객체
Movement	지면, 대시, 속도 등 이동 상태
AimDirection	현재 동기화된 조준 방향
AimAngle	현재 동기화된 조준 각도
AttackOrigin	근접 또는 범위 공격 기준 Transform
ProjectileOrigin	투사체 생성 기준 Transform
EffectOrigin	VFX 생성 기준 Transform
WeaponRoot	무기 비주얼의 부모 Transform

쿨타임은 **Cooldowns.GetRemaining(action)**으로 남은 시간을, **Cooldowns.GetDuration(action)**으로 전체 시간을 조회할 수 있다. 기본 스킬 쿨타임 시작은 Base가 처리하므로 캐릭터 스크립트에서 다시 시작하지 않는다.

6.7 패시브 이벤트 API

API	호출 시점	전달되는 값
OnPassiveTick(deltaTime)	Simulation 틱마다	틱 시간
OnDamaged(damage)	자신이 피해를 받은 뒤	요청 피해, 실제 피해, 공격자
OnDamageDealt(target, requestedDamage)	다른 캐릭터에게 피해 요청 후	대상과 요청 피해량
OnDied(attacker)	사망했을 때	마지막 공격자
OnJumped()	점프했을 때	없음
OnLanded()	착지했을 때	없음
OnSkillExecuted(action)	행동 실행에 성공했을 때	행동 종류
OnHealthChanged(previous, current)	체력이 변경됐을 때	변경 전후 체력
OnCharacterSpawned()	네트워크 캐릭터 생성 완료	없음
OnResetCharacter()	캐릭터 초기화	없음

CharacterDamageInfo.RequestedDamage는 공격자가 요청한 피해량이고, **AppliedDamage**는 체력에 실제로 반영된 피해량이다. 방어, 무적, 남은 체력의 영향까지 고려해야 하면 **AppliedDamage**를 사용한다.

이벤트로 표현할 수 있는 패시브는 **OnPassiveTick**으로 반복 검사하지 않는다. 피격 패시브는 **OnDamaged**, 착지 패시브는 **OnLanded**, 스킬 성공 패시브는 **OnSkillExecuted**에 구현한다.

7. 신규 캐릭터 개발 절차

1단계: 기획 명세 작성

구현 전에 아래 항목을 먼저 채운다.

캐릭터 이름:
 역할:
 핵심 전투 특성:
 평타:
 Q:
 E:
 대시:
 궁극기:
 패시브:
 필요한 투사체 또는 설치물:
 공통 API만으로 구현하기 어려운 부분:

2단계: 템플릿과 데이터 생성

1. CharacterTemplate.cs를 복사한다.
2. 클래스와 파일 이름을 캐릭터 이름으로 변경한다.
3. CharacterDefinition을 생성하고 기본 수치를 입력한다.
4. 기본 Visual Profile을 사용할지 전용 Profile이 필요한지 결정한다.

3단계: 프리팹 Variant 구성

1. 공통 Character Template Prefab의 Variant를 생성한다.
2. 캐릭터 스크립트와 Character Definition을 연결한다.
3. Body와 SpriteRenderer를 연결한다.
4. Attack Origin을 공격 기준 위치에 배치한다.
5. 필요한 경우에만 Projectile Origin, Weapon Root, Effect Origin을 연결한다.
6. CharacterVisualController에 Body와 필요한 Visual 대상을 등록한다.

4단계: 기능을 하나씩 구현

다음 순서를 권장한다.

1. 이동과 조준 확인
2. 평타 구현
3. 대시 구현 또는 기본 대시 사용
4. Q 구현

5. E 구현
6. 궁극기 구현
7. 패시브 구현
8. VFX와 사운드 연결
9. 밸런스 수치 조정

한 기능을 구현한 뒤 판정과 쿨타임을 확인하고 다음 기능으로 넘어간다. 여러 기능을 한꺼번에 작성하면 중복 실행이나 잘못된 판정의 원인을 찾기 어렵다.

5단계: 자동 구성 검사

캐릭터 프리팹을 선택한 뒤 다음 메뉴를 실행한다.

Tools > Project MS > Character > Validate Selected Character

Validator가 다음 필수 구성을 확인한다.

CharacterBase를 상속한 컴포넌트
 NetworkObject
 Rigidbody2D와 Collider2D
 Character Definition
 CharacterVisualController

6단계: Fusion 2인 테스트

Shared Mode에서 두 클라이언트를 실행하고 로컬 캐릭터와 원격 캐릭터 양쪽을 확인한다. Unity가 없는 환경에서는 코드 구조와 금지 API만 정적으로 검사하고, 이 단계는 Unity가 있는 환경에서 최종 진행한다.

8. 캐릭터 개발자가 지킬 네트워크 경계

캐릭터별 스크립트에는 원칙적으로 다음 코드를 작성하지 않는다.

Update, FixedUpdate, FixedUpdateNetwork, Render

Networked 프로퍼티

Rpc 메서드

Runner.Spawn

HasStateAuthority 또는 HasInputAuthority 검사

Rigidbody2D를 이용한 별도의 비주얼 이동

스킬 클래스는 `SpawnProjectile`, `FindEnemies...`, `DealDamage`, `StartDash`, `PlayActionEffect` 같은 공통 API를 사용한다. 공통 API만으로 구현할 수 없는 기능은 캐릭터 클래스에 Fusion 코드를 추가하지 말고 공통 시스템 담당자에게 필요한 동작을 전달한다.

공통 시스템 담당자에게 요청할 정보

기능이 시작되고 끝나는 조건

모든 클라이언트가 알아야 하는 상태

중간 입장자가 복원해야 하는 상태

생성되는 `NetworkObject`의 종류와 수명

피해가 적용되는 시점과 횟수

취소, 사망, 재접속 시 정리 규칙

충전, 채널링, 변신, 지속 장판, 설치물, 소환수처럼 여러 틱 동안 상태가 유지되는 기능은 별도 공통 네트워크 모듈이 필요할 수 있다.

9. 검수 체크리스트

프리팍과 데이터

- Character Definition이 연결되어 있는가?
- 필수 컴포넌트가 모두 있는가?
- Attack Origin이 올바른 위치에 있는가?
- 사용하지 않는 선택 Socket을 억지로 추가하지 않았는가?
- CharacterVisualController에 Body가 연결되어 있는가?

스킬과 패시브

- 한 번의 입력에 행동이 한 번만 실행되는가?
- 성공한 행동에만 쿨타임이 적용되는가?
- 데미지가 한 대상에게 중복 적용되지 않는가?
- 자기 자신이나 소유자를 공격하지 않는가?
- 패시브가 적절한 이벤트 함수에 구현되어 있는가?

네트워크

- 각 플레이어가 자기 캐릭터만 조작하는가?
- 로컬과 원격에서 공격 방향과 결과가 일치하는가?
- 투사체와 효과가 중복 생성되지 않는가?
- 체력과 사망 상태가 양쪽에서 일치하는가?
- 중간 입장자가 현재 상태를 올바르게 보는가?

비주얼

- Visual 코드가 입력을 직접 읽지 않는가?
- Visual 코드가 Rigidbody2D 상태를 변경하지 않는가?
- 점프, 착지, 피격 표현이 원격에서도 보이는가?
- 이름표와 WorldCanvas가 뒤집히지 않는가?

10. 완료 기준

신규 캐릭터는 다음 조건을 모두 만족하면 완료로 본다.

공통 프리팹 Variant와 Character Definition을 사용한다.

캐릭터별 클래스에는 스킬과 패시브 규칙만 들어 있다.

캐릭터별 클래스가 Fusion API를 직접 사용하지 않는다.

무기 방식과 관계없이 CharacterBase의 공통 판정 API를 사용한다.

추가 Visual MonoBehaviour 없이 CharacterVisualController 설정으로 표현한다.

자동 Validator에서 필수 구성 오류가 없다.

Fusion Shared Mode 2인 테스트를 통과한다.

이 구조의 최종 목표는 신규 캐릭터 개발자가 네트워크 생명주기를 몰라도 캐릭터의 고유한 전투 방식과 재미를 안전하게 구현할 수 있게 하는 것이다.